Documento de diseño del proyecto NanoFilesP2P

- REDES DE COMUNICACIONES -

- 24/25 -

- **Jorge Torralba Santa Cruz**

DNI: CENSURADO
Subgrupo: 1.3

- **Guillermo Fulgencio Parra López**

DNI: CENSURADO
Subgrupo: 1.3

Índice

Índice **2**
1 Introducción **4**
1.1 Componentes del sistema 4
1.2 Objetivos principales 4
1.3 Estructura de la memoria 4
2 Formato de los mensajes del protocolo de comunicación con el Directorio **5**
2.1 Mensaje ping 5
2.2 Mensaje pingOk 5
2.3 Mensaje pingBad 6
2.4 Mensaje filelistRequest 6
2.5 Mensaje filelistResponseOk 6
2.6 Mensaje filelistResponseBad 7
2.7 Mensaje serveRequest 7
2.8 Mensaje serveOk 7
2.9 Mensaje serveBad 8
2.10 Mensaje downloadRequest 8
2.11 Mensaje downloadResponseOk 8
2.11 Mensaje downloadResponseAmbiguous 9
2.13 Mensaje downloadResponseNotFound 9
2.14 Mensaje quit 9
2.15 Mensaje quitOk 10
2.16 Mensaje quitBad 10
2.17 Mensaje userlistRequest 10
2.18 Mensaje userlistResponseOk 11
2.18 Mensaje userlistResponseBad 11
3 Formato de los mensajes del protocolo de transferencia de ficheros **12**
3.0 Mejora añadida, encriptado 12
3.1 Mensaje DownloadFile 12
3.2 Mensaje FileMetaData 12
3.3 Mensaje AmbiguousName 13
3.4 Mensaje FileNotFound 13
3.5 Mensaje ChunkRequest 13
3.6 Mensaje ChunkRequest 13
3.7 Mensaje EndOfFile 14
3.8 Mensaje ChunkNotFound 14
3.9 Mensaje UploadRequest 14
3.10 Mensaje UploadAccepted 14
3.11 Mensaje UploadRejected 15
3.12 Mensaje UploadData 15
3.13 Mensaje UploadComplete 15
4 Autómatas de protocolo **16**
4.1 Autómata rol cliente de directorio 16

4.2 Autómata rol servidor de directorio 17
4.3 Autómata rol cliente de ficheros 17
4.4 Autómata rol servidor de ficheros 18
5 Mejoras implementadas y breve descripción sobre su programación 19
5.1 Comando upload 19
5.2 Comando serve con puerto efímero 19
5.3 Comando filelist ampliado 19
5.4 Comando quit actualiza ficheros y servidores 19
5.5 Comando download paralelo 20
5.6 Comando userlist 20
5.7 Secure P2P 21
5.8 Enhanced user experience 21
5.9 Custom chunks 21
5.10 Close sockets 21
6 Capturas de pantalla que muestren mediante Wireshark un intercambio de mensajes con el directorio 22
7 Enlace a grabación de pantalla mostrando los programas en funcionamiento 22
8 Conclusiones 22
8.1 Aprendizaje técnico 22
8.2 Desafíos encontrados 22
8.3 Resultados obtenidos 23
8.4 Limitaciones y líneas de trabajo futuro 23
8.5 Valoración personal 23

1 Introducción

Este proyecto se centra en el diseño e implementación de nanoFiles, un sistema de compartición y transferencia de archivos desarrollado para la asignatura de Redes de Comunicaciones, convocatoria de Mayo, curso 2024/25. El sistema combina elementos de arquitecturas cliente-servidor y peer-to-peer (P2P), permitiendo a los usuarios compartir y descargar archivos de manera eficiente.

1.1 Componentes del sistema

El sistema nanoFiles consta de dos elementos principales:

- Servidor de directorio (Directory): Mantiene información centralizada sobre los archivos compartidos y los servidores que los ofrecen. La comunicación con este componente se realiza mediante un protocolo fiable implementado sobre UDP.
- Peers (NanoFiles): Actúan como clientes y/o servidores, permitiendo consultar el directorio, descargar archivos de otros peers y compartir archivos propios con el resto de usuarios del sistema.

1.2 Objetivos principales

- Diseñar un protocolo fiable sobre UDP para la comunicación con el directorio.
- Diseñar un protocolo eficiente sobre TCP para la transferencia de archivos entre peers.
- Implementar la funcionalidad básica del sistema: consulta de archivos disponibles, publicación de archivos compartidos y descarga de archivos desde múltiples servidores.
- Desarrollar funcionalidades adicionales como la transferencia bidireccional de archivos y la actualización dinámica del directorio.

1.3 Estructura de la memoria

Esta memoria incluye la descripción detallada de los protocolos diseñados, el formato de los mensajes intercambiados, los autómatas que modelan el comportamiento de los componentes, las mejoras implementadas y las conclusiones obtenidas durante el desarrollo del proyecto.

2 Formato de los mensajes del protocolo de comunicación con el Directorio

Para definir el protocolo de comunicación con el *Directorio*, vamos a utilizar mensajes textuales con formato “campo:valor”. El valor que tome el campo “operation” (código de operación) indicará el tipo de mensaje y por tanto su formato (qué campos vienen a continuación).
Los ejemplos pueden verse en el apartado de Wireshark.

2.1 Mensaje ping

Operación: ping.
Emisor: NanoFiles.
Receptor: Directory.
Campos adicionales: identificador de protocolo del peer.
Finalidad: pedirle al directorio si está activo y si el id. de protocolo es compatible.
Condiciones de envío: ninguna.
Acciones al recibirlo: el directorio compara el id. protocolo recibido por el suyo, y responde consecuentemente.

- **Formato**

  operation: ping\n
  protocol: <protocolId>\n
  \n

2.2 Mensaje pingOk

Operación: pingOk.
Emisor: Directory.
Receptor: NanoFiles.
Campos adicionales: ninguno.
Finalidad: informar al peer que el directorio está a la escucha y el id. protocolo es compatible.
Condiciones de envío: se acaba de recibir un mensaje “ping” y se ha comprobado la compatibilidad.
Acciones al recibirlo: el peer informa por la pantalla que el directorio está activo y es compatible.

- **Formato**

  operation: pingOk\n
  \n

2.3 Mensaje pingBad

Operación: pingBad
Emisor: Directory.
Receptor: NanoFiles.
Campos adicionales: ninguno.
Finalidad: informar al peer de que de que el directorio está a la escucha pero el id. protocolo es Incompatible.
Condiciones de envío: se acaba de recibir un mensaje “ping” y se ha comprobado la Incompatibilidad.
Acciones al recibirlo: el peer informa por la consola que el directorio está activo y NO es compatible.

- **Formato**

  operation: pingBad\n
  \n

2.4 Mensaje filelistRequest

Operación: filelistRequest.
Emisor: NanoFiles.
Receptor: Directory.
Campos adicionales: ninguno.
Finalidad: solicita al servidor de ficheros la lista de sus archivos disponibles.
Condiciones de envío: estar conectado con el directorio.
Acciones al recibirlo: comprueba la lista de ficheros disponibles.

- **Formato**

operation: filelistRequest\n
  \n

2.5 Mensaje filelistResponseOk

Operación: filelistResponseOk.
Emisor: Directory.
Receptor: Nanofiles.
Campos adicionales: array de ficheros que están compartidos en el directorio.
Finalidad: responder al mensaje “filelistRequest” con el array de ficheros que están compartidos en el directorio.
Condiciones de envío: acaba de recibir un mensaje “filelistRequest”.
Acciones al recibirlo: se muestran por pantalla los ficheros compartidos en el directorio.

- **Formato**

  operation: filelistResponseOk\n
  num archivos: <num archivos>\n
  archivo: <archivo><Conjunto de servidores que lo ofrecen>\n
  …
  archivo: <archivo><Conjunto de servidores que lo ofrecen>\n
  \n

2.6 Mensaje filelistResponseBad

Operación: filelistResponseBad.
Emisor: Directory.
Receptor: Nanofiles.
Campos adicionales: ninguno.
Finalidad: advertir de que no hay ficheros compartidos en el directorio.
Condiciones de envío: acaba de recibir un mensaje “filelistRequest” y no hay ficheros compartidos en el directorio.
Acciones al recibirlo: se muestran por pantalla advirtiendo de que no hay ficheros compartidos en el directorio.

- **Formato**

  operation: filelistResponseBad\n
  \n

2.7 Mensaje serveRequest

Operación: serveRequest.
Emisor: NanoFiles.
Receptor: Directory.
Campos adicionales: ficheros que va a compartir el peer (y número) y el puerto.
Finalidad: compartir en el directorio los ficheros de un peer.
Condiciones de envío: estar conectado con el directorio.
Acciones al recibirlo: incluye los ficheros en el directorio y se devuelve el mensaje de respuesta correspondiente.

- **Formato**

  operation: serveRequest\n
  puerto: <puerto>\n
  num archivos: <num archivos>\n
  archivo: <archivo>\n
  …
  archivo: <archivo>\n
  \n

2.8 Mensaje serveOk

Operación: serveOk.
Emisor: Directory.
Receptor: NanoFiles.
Campos adicionales: ninguno.
Finalidad: informar al peer de que la operación “serve” se ha realizado con éxito.
Condiciones de envío: acaba de recibir un mensaje “serve” y coincide el puerto.
Acciones al recibirlo: muestra por pantalla que tus archivos se han subido correctamente.

- **Formato**

  operation: serveOk\n
  \n

2.9 Mensaje serveBad

Operación: serveBad.
Emisor: Directory
Receptor: Nanofiles.
Campos adicionales: ninguno.
Finalidad: informar de que el puerto no coincide y, por tanto, no se ha podido realizar la operación “serve”.
Condiciones de envío: acaba de recibir un mensaje “serve” y el puerto no coincide.
Acciones al recibirlo: muestra por pantalla que no se han podido subir los archivos.

- **Formato**

  operation: serveBad\n
  \n

2.10 Mensaje downloadRequest

Operación: downloadRequest.
Emisor: NanoFiles.
Receptor: Directory.
Campos adicionales: subcadena del nombre del fichero.
Finalidad: solicitar las direcciones de los peers que están compartiendo el fichero buscado.
Condiciones de envío: estar conectado con el directorio.
Acciones al recibirlo: comprobar que existe un fichero que contiene en su nombre dicha subcadena y enviar la cadena correspondiente.

- **Formato**

  operation: downloadRequest\n
  cadena: <subcadena fichero>\n
  \n

2.11 Mensaje downloadResponseOk

Operación: downloadResponseOk
Emisor: Directory.
Receptor: NanoFiles.
Campos adicionales:InetSocketAddress de los peers que comparten el fichero buscado.
Finalidad: informar al peer de que se ha encontrado el fichero y enviarle las direcciones de los peers que lo comparten.
Condiciones de envío: se acaba de recibir un mensaje “downloadRequest” y se ha encontrado un único fichero que contiene la subcadena pasada por el NanoFiles.
Acciones al recibirlo: muestra por pantalla las direcciones de los peers que ofrecen el fichero buscado.

- **Formato**

  operation: downloadResponseOk\n
  num direcciones: <número de direcciones>\n
  direccion: <direccion peer>\n
  …

direccion: <direccion peer>\n
\n

2.11 Mensaje downloadResponseAmbiguous

Operación: downloadResponseAmbiguous.
Emisor: Directory.
Receptor: NanoFiles.
Campos adicionales: añadimos un nuevo campo que indica el número de ficheros encontrados que contienen la subcadena.
Finalidad: informar de que la subcadena pasada no es suficientemente exacta como para saber el fichero buscado.
Condiciones de envío: se acaba de recibir un mensaje “downloadRequest” y se han encontrado dos o más ficheros que contienen la subcadena pasada por NanoFiles.
Acciones al recibirlo: advierte por pantalla que la subcadena pasada es ambigua.

- **Formato**

  operation: downloadResponseAmbiguous\n
  coincidencias: <número de coincidencias>\n
  \n

2.13 Mensaje downloadResponseNotFound

Operación: downloadResponseNotFound.
Emisor: Directory.
Receptor: NanoFiles.
Campos adicionales: ninguno.
Finalidad: informar al peer de que no se ha encontrado ningún fichero que contenga la subcadena recibida.
Condiciones de envío: se acaba de recibir un mensaje “downloadRequest” y no se ha encontrado ningún fichero que contenga la subcadena enviada por NanoFiles.
Acciones al recibirlo: advierte por pantalla que ningún fichero contiene la subcadena pasada.

- **Formato**

  operation: downloadResponseNotFound\n
  \n

2.14 Mensaje quit

Operación: quit
Emisor: NanoFiles.
Receptor: Directory.
Campos adicionales: ninguno.
Finalidad: el peer solicita desconectarse del directorio y dejar de compartir sus ficheros.
Condiciones de envío: debe estar conectado.
Acciones al recibirlo: el directorio elimina al peer de su base de datos, así como deja de ofrecer los ficheros que este haya subido. Si todo va bien, responde con quitOk, si no, responde quitBad.

- **Formato**

  operation: quit\n
  \n

2.15 Mensaje quitOk

Operación: quitOk
Emisor: Directory.
Receptor: NanoFiles.
Campos adicionales: ninguno.
Finalidad: confirmar que el peer ha sido eliminado correctamente del directorio.
Condiciones de envío: el directorio ha recibido un `quit` y ha eliminado correctamente la información del peer.
Acciones al recibirlo: el peer muestra un mensaje indicando que la desconexión se ha realizado con éxito.

- **Formato**

  operation: quitOk\n
  \n

2.16 Mensaje quitBad

Operación: quitBad
Emisor: Directory.
Receptor: NanoFiles.
Campos adicionales: ninguno.
Finalidad: indicar que no se ha podido eliminar correctamente al peer del directorio (por ejemplo, si el peer no estaba registrado o hubo un error interno).
Condiciones de envío: el directorio ha recibido un quit pero no ha podido procesarlo correctamente.
Acciones al recibirlo: el peer muestra un error indicando que la operación de desconexión ha fallado.

- **Formato**

  operation: quitBad\n
  \n

2.17 Mensaje userlistRequest

Operación: userlistRequest
Emisor: NanoFiles.
Receptor: Directory.
Campos adicionales: ninguno.
Finalidad: solicitar al directorio la lista de usuarios conectados y sus archivos compartidos.
Condiciones de envío: estar conectado con el directorio.
Acciones al recibirlo: el directorio recopila la información de los usuarios conectados y sus archivos compartidos, y responde con un mensaje userlistResponseOk.

- **Formato**

  operation: userlistRequest\n
  \n

2.18 Mensaje userlistResponseOk

Operación: userlistResponseOk.
Emisor: Directory.
Receptor: NanoFiles.
Campos adicionales: lista de usuarios conectados con sus direcciones y archivos compartidos.
Finalidad: responder al mensaje “userlistRequest” con la información de los usuarios conectados y sus archivos compartidos.
Condiciones de envío: acaba de recibir un mensaje “userlistRequest”.
Acciones al recibirlo: NanoFiles muestra por pantalla la lista de usuarios conectados y sus archivos compartidos.

- **Formato**

  operation: userlistResponseOk\n
  userlist: <lista_serializada_de_usuarios>\n
  \n

Nota: El campo userlist contiene una representación serializada de un mapa donde cada usuario está identificado por su dirección (host:puerto), asociada a un conjunto de archivos compartidos (nombres de archivos).

2.18 Mensaje userlistResponseBad

Operación: userlistResponseBad.
Emisor: Directory.
Receptor: NanoFiles.
Campos adicionales: ninguno
Finalidad: responder al mensaje “userlistRequest” con que algo ha ido mal.
Condiciones de envío: acaba de recibir un mensaje “userlistRequest”.
Acciones al recibirlo: nada.

- **Formato**

  operation: userlistResponseBad\n
  \n

3 Formato de los mensajes del protocolo de transferencia de ficheros

3.0 Mejora añadida, encriptado

Hemos decidido añadir una mejora que no figuraba entre las disponibles. Esta consiste en encriptar los mensajes de comunicación P2P, para una mayor privacidad. Por ello, y haciendo una analogía con la terminología de “muñecas

rusas” vista en teoría, los mensajes de nuestro protocolo de nivel de aplicación quedan de la siguiente forma. Esto es debido a que los métodos de la clase que maneja la encriptación devuelven arrays de bytes, que deberemos declarar con un tamaño concreto, length.

length	Mensaje
4 bytes	n bytes

3.1 Mensaje DownloadFile

Mensaje: DownloadFile (opcode = 1)
Sentido de la comunicación: Cliente → Servidor de ficheros
Descripción: Este mensaje lo envía el par cliente al par servidor de ficheros para enviar una petición de descarga de fichero con dicho nombre.

Opcode	longFileName	fileName
0x01	1 byte	n bytes

3.2 Mensaje FileMetaData

Mensaje: FileMetData (opcode = 2)
Sentido de la comunicación: Servidor de ficheros → Cliente
Descripción: Este mensaje lo envía el par servidor al par cliente de ficheros para enviar el fichero.

Opcode	fileSize	fileHash
0x02	8 bytes	40 bytes

3.3 Mensaje AmbiguousName

Mensaje: AmbiguousName (opcode = 3)
Sentido de la comunicación: Servidor de ficheros → Cliente
Descripción: Este mensaje lo envía el par servidor al par cliente de ficheros para indicar que el nombre del fichero que se desea descargar es ambiguo.

Opcode
0x03

3.4 Mensaje FileNotFound

Mensaje: FileNotFound (opcode = 4)
Sentido de la comunicación: Servidor de ficheros → Cliente
Descripción: Este mensaje lo envía el par servidor de ficheros al par cliente (receptor) de fichero para indicar que no es posible encontrar el fichero con la información proporcionada en el mensaje de petición de descarga.

Opcode
0x04

3.5 Mensaje ChunkRequest

Mensaje: ChunkRequest (opcode = 5)
Sentido de la comunicación: Cliente → Servidor de ficheros
Descripción: Este mensaje lo envía el cliente para solicitar un fragmento (chunk) de un fichero concreto, identificándolo por su hash.

Opcode	fileHash	chunkInicio	chunkFin
0x05	40 bytes	4 bytes	4 bytes

3.6 Mensaje ChunkRequest

Mensaje: ChunkResponse (opcode = 6)
Sentido de la comunicación: Servidor de ficheros → Cliente
Descripción: Este mensaje lo envía el servidor para entregar al cliente el fragmento solicitado del fichero.

Opcode	posicionChunk	chunkLength	chunkData
0x06	4 bytes	4 bytes	n bytes

3.7 Mensaje EndOfFile

Mensaje: EndOfFile (opcode = 7)
Sentido de la comunicación: Servidor de ficheros → Cliente
Descripción: Este mensaje lo envía el servidor para indicar que el fragmento solicitado está fuera del rango del fichero.

Opcode
0x07

3.8 Mensaje ChunkNotFound

Mensaje: ChunkNotFound (opcode = 8)
Sentido de la comunicación: Servidor de ficheros → Cliente
Descripción: Este mensaje lo envía el servidor cuando no encuentra el fragmento del fichero que se le ha solicitado.

Opcode
0x08

3.9 Mensaje UploadRequest

Mensaje: UploadRequest (opcode = 9))
Sentido de la comunicación: Cliente → Servidor de ficheros
Descripción: Este mensaje lo envía el cliente para solicitar permiso al servidor para subir un fichero.

Opcode	fileSize	fileHash	longFileName	fileName
0x09	8 bytes	40 byte	1 byte	n bytes

3.10 Mensaje UploadAccepted

Mensaje: UploadAccepted (opcode = 10)
Sentido de la comunicación: Servidor de ficheros → Cliente
Descripción: Este mensaje lo envía el servidor aceptando la solicitud de subida del fichero.

Opcode
0x0A

3.11 Mensaje UploadRejected

Mensaje: UploadRejected (opcode = 11)
Sentido de la comunicación: Servidor de ficheros → Cliente
Descripción: Este mensaje lo envía el servidor rechazando la subida del fichero, generalmente porque ya existe.

Opcode
0x0B

3.12 Mensaje UploadData

Mensaje: UploadData (opcode = 12)
Sentido de la comunicación: Cliente → Servidor de ficheros
Descripción: Este mensaje lo envía el cliente para transferir un fragmento de datos del fichero a subir.

Opcode	posicionChunk	chunkLength	chunkData
0x0C	4 bytes	4 bytes	n bytes

3.13 Mensaje UploadComplete

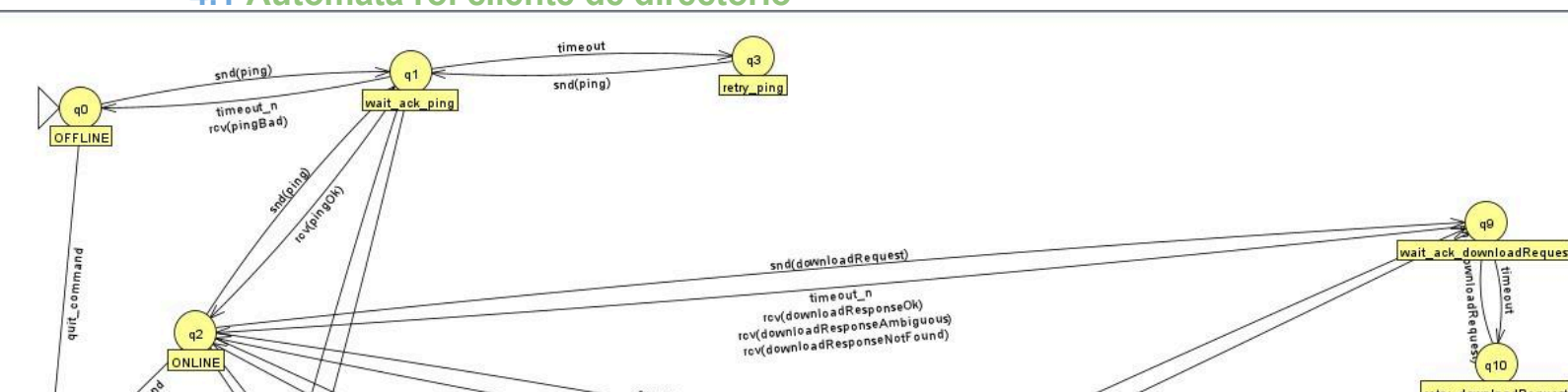
Mensaje: UploadComplete (opcode = 13)
Sentido de la comunicación: Cliente → Servidor de ficheros
Descripción: Este mensaje lo envía el cliente para indicar que ha finalizado la transferencia del fichero.

Opcode
0x0D

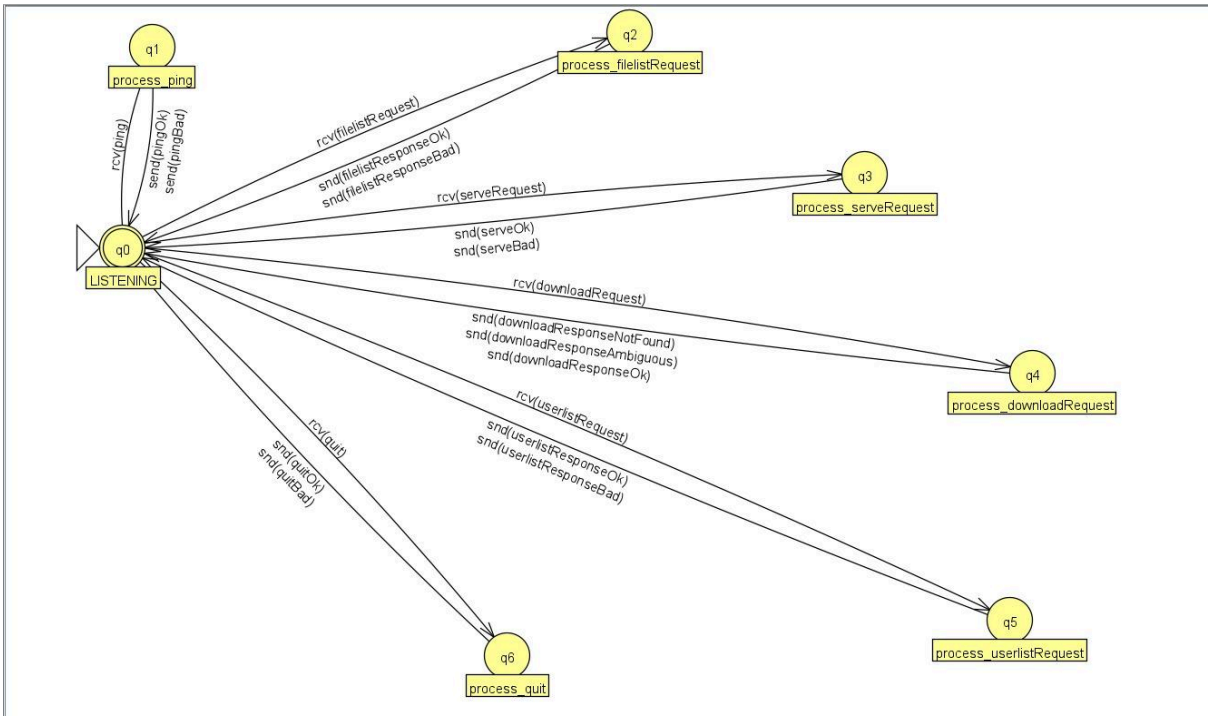
4 Autómatas de protocolo

En el caso de rol cliente de directorio los estados de autómata se actualizan consecuentemente en el código.

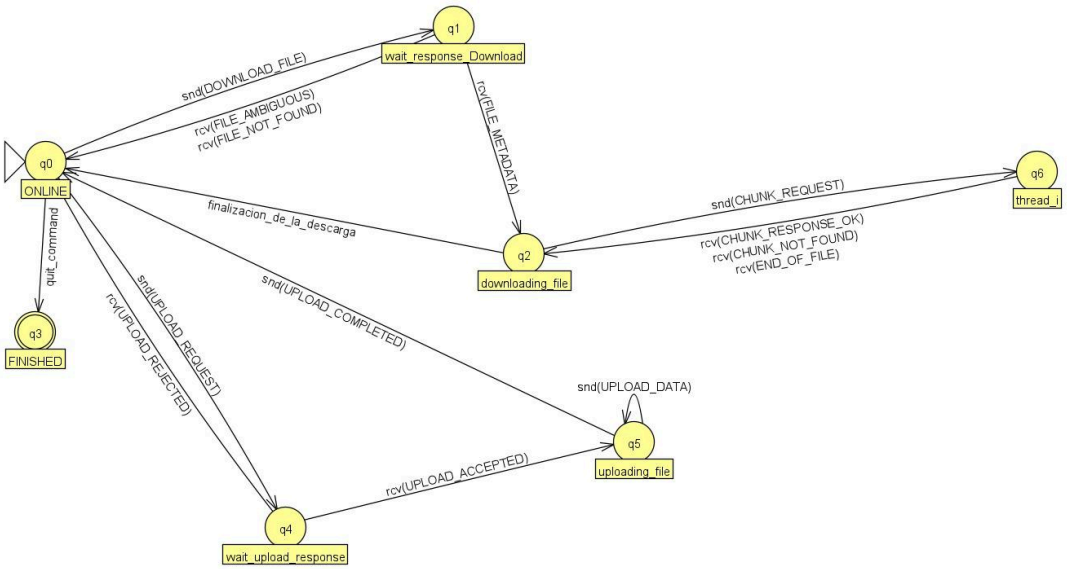
4.1 Autómata rol cliente de directorio



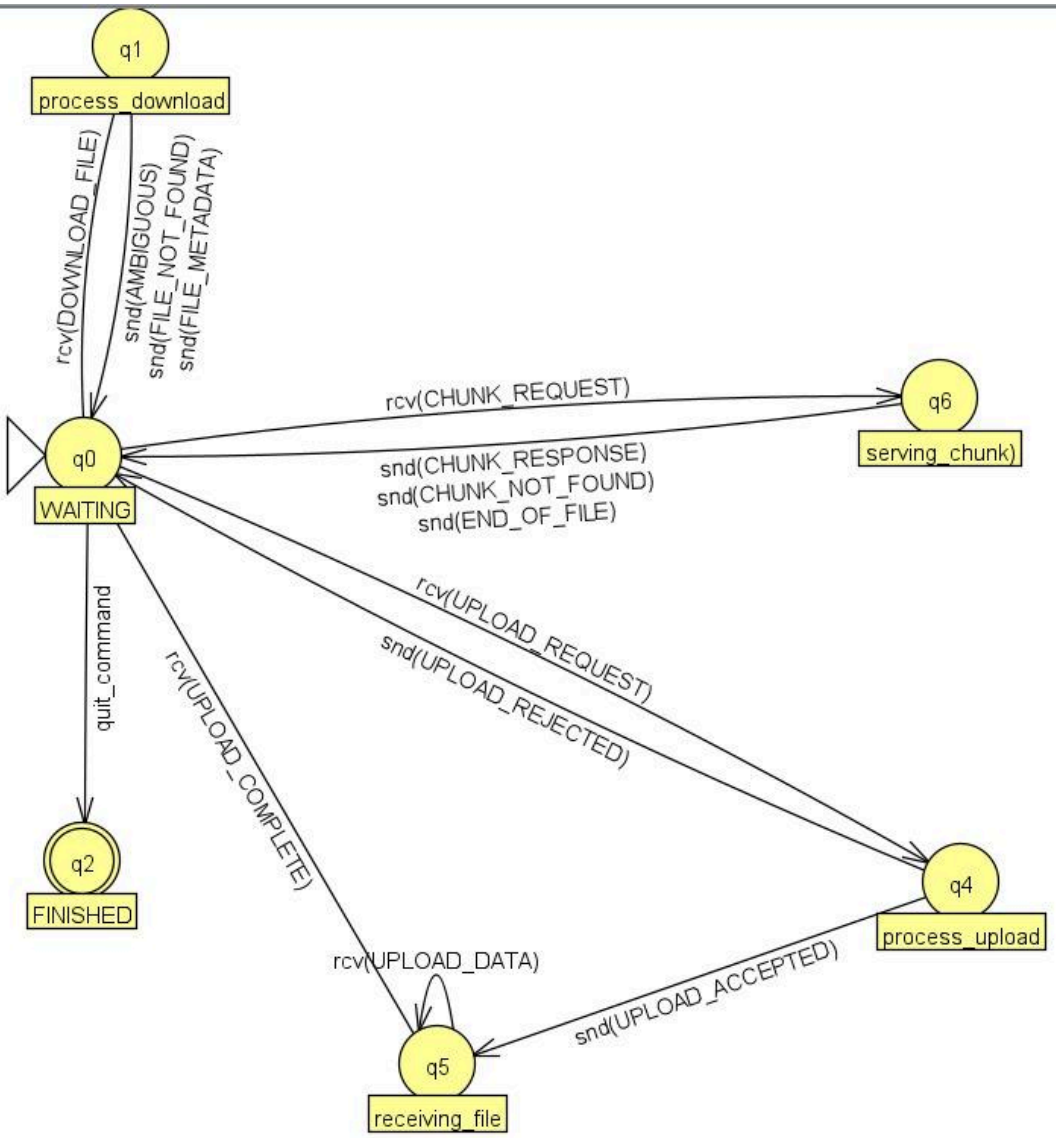
4.2 Autómata rol servidor de directorio



4.3 Autómata rol cliente de ficheros



4.4 Autómata rol servidor de ficheros



5 Mejoras implementadas y breve descripción sobre su programación

5.1 Comando upload

El cliente comprueba que disponga del archivo para posteriormente pedirle al peer subirlo. Si este acepta, leemos el archivo de chunk en chunk enviándolo SECUENCIALMENTE (posible línea de trabajo futura, ampliarlo a paralelo). Cuando la hemos completado acabamos para que el servidor la coja de la estructura concurrente (concurrente ya que pueden subir varios clientes a la vez diferentes archivos) a la que ha ido metiendo los chunks conforme los recibía. El servidor actualiza su base de datos y finaliza la subida.

5.2 Comando serve con puerto efímero

Una mejora bastante sencilla (y necesaria si quieres probar a descargar un fichero de varios peers en localhost), la cual hemos implementado realizando modificaciones triviales en la clase encargada de leer los comandos para permitir que *serve* reciba en un argumento el puerto. Hemos añadido algunas comprobaciones para evitar que se reciba un puerto no a lugar. Finalmente, basta con añadir un constructor a *NFServer*, que modifique el atributo puerto, también añadido. Y usar este atributo al construir la dirección a la que se *bindeará* el socket.

5.3 Comando filelist ampliado

Basta con añadir una estructura (mapa) que asocie a cada fichero un conjunto de ips que lo han servido. Esta se actualizará en cada serve (y quit, mejora de abajo). En cuanto a su paso por mensajes hemos reutilizado el atributo que emplea userlist, se explica más abajo. No obstante, se trata línea a línea.

5.4 Comando quit actualiza ficheros y servidores

Si se recibe un quit, en caso de que el nanofiles que lo ha ejecutado esté sirviendo, mandará un mensaje al directorio para actualizar todas las estructuras de datos que lo involucren, eliminandolo de forma pertinente.

Pensamos en mejorar más aún esta mejora y que también se actualizasen en caso de muerte súbita, realizando pings o isAlive cada ciertos segundos a todos los peers para ver si siguen vivos. No obstante, la falta de tiempo no nos ha permitido recrearnos tanto.

5.5 Comando download paralelo

Esta mejora la hemos realizado creando una nueva clase. Esta clase nos ayuda a encapsular un poco la lógica (evitando así que el método de descargar sea más grande aún). En la susodicha nos conectamos en primer lugar al servidor que se nos ha asignado y en caso de que no sea posible iteramos sobre el resto, hasta un máximo de 3 veces. Almacenamos en un mapa concurrente de que servidor se ha bajado el chunk cada hilo. El chunk se escribe en un array de bytes que se accede en exclusión mutua (cerrojo lock), a su vez se registra el éxito en un array de booleanos, también accedido de forma concurrente. Lo de que los chunks se escriban en un array es algo simple que tiene sus limitaciones (el tamaño del array viene dado por un int, por lo que tenemos que nos reducimos a 2GiB de subida máxima, siendo una línea de trabajo futura el uso de RandomAccessFile para salvar esto). Otra limitación es el número máximo de hilos, que para evitar saturación de la CPU (así como que el firewall rechace algunas conexiones) lo hemos dejado en un máximo de 4 por servidor o 4 por hilo que tenga nuestra CPU (línea de trabajo futura, hacer una pool de hilos con executor).

Los hilos se van asignando a direcciones por roundrobin, para asegurar un reparto equilibrado.

Cuando todos los hilos terminan se comprueba si todos los chunks se han descargado con exito (con el array de booleanos). También se comprueba el hash, se imprime alguna información de interés, guardando el archivo y actualizando la base de datos de ficheros.

5.6 Comando userlist

Recibiendo la idea de la Rúbrica de corrección del Aula Virtual decidimos hacer esta mejora. Para ella usaremos una estructura de mapa que asociada a cada dirección el conjunto de ficheros que está sirviendo. Si además queremos añadir aquellos usuarios que no sirven ficheros necesitaremos un mapa que asocie a cada usuario conectado al directorio (su IP): nulo en caso de que no esté sirviendo o la IP con la que lo hace en caso contrario. A la hora de pasar por tanto esta información en un mensaje usamos una estructura (mapa) que asocia la IP con la que hace de cliente de directorio (y de otros peers) con otro mapa (que tendrá una sola entrada, pero queriamos evitar trabajar con Map.Entry, y Java no permite structs…) el cual, a su vez, asocia la IP con la que sirve ficheros (actuando de peer servidor) con el conjunto de ficheros que sirve. Para pasar de string al mapa y viceversa usamos una clase (creada ad-hoc) que dispone de ambos métodos y se hace todo en una linea. Por tanto, por un lado, tenemos un nuevo enfoque y modo de pasar mensajes textuales el cual es usado más en la actualidad (sobre todo gracias a JSON) siendo su principal desventaja un detrimento en la legibilidad el cual se salva bastante bien en esta ocasión (véase ejemplo de mensaje userlistResponse). El otro enfoque sería ir línea por línea, este también está presente en nuestro proyecto, por ejemplo en el downloadResponse, filelistResponse o serveRequest.

5.7 Secure P2P

Para tener un protocolo además de confiable, seguro; hemos decidido que nuestros mensajes viajen encriptados. La verdad que ha supuesto muy pocos cambios ya que solo tenemos que alterar como se guardan los mensajes en el *dis* y *dos*. Esto permitirá que si alguien sniffa el tráfico con *Wireshark* y lo vierte *okteta* no pueda ver el contenido de nuestros ficheros. Encriptamos usando librerías ya proporcionadas por Java, con un algoritmo AES y una sencilla clase que hemos creado.

5.8 Enhanced user experience

Consideramos que este proyecto, al ser un protocolo de aplicación, debe mostrar solo por pantalla (salvo que se modifiquen las constantes adecuadas, para usuarios expertos) lo estrictamente necesario para un usuario de a pie que quiera usarla. Ocultándose al máximo posible la lógica, el intercambio de mensajes, la concurrencia…

Además, la salida por pantalla ha sido mejorada significativamente con ASCII box.

Disclaimer: la versión entregada (jars) será con los mensajes de debuggeo ya que habrá que demostrar la concurrencia.

5.9 Custom chunks

Pensamos que un tamaño de chunks de 1KB es, permítanos la expresión, tercermundista, por ello se permite al usuario seleccionar el tamaño deseado para sus subidas y descargas. No obstante esto vendrá limitado por la capacidad de nuestra CPU para crear hilos.

5.10 Close sockets

Bajo nuestro punto de vista la calidad del software, así como la robustez y reusabilidad del código son claves. De ahí que tratemos de cerrar siempre todos los sockets que se crean (aunque al cerrar el programa Java lo haga de por sí, invocando System.exit(), no obstante, es buena práctica cerrarlos). En particular, hemos hecho modificaciones para cerrar el que usa un nanofiles para comunicarse con el directorio. Como también para cerrar los diferentes que se crean al descargar o subir archivos a un peer servidor. Para esto último, hemos optado por hacer que NFConnector extienda a autocloseable y usar el maravilloso recurso que nos regaló Java 7 conocido como try-with-resources.

6 Capturas de pantalla que muestren mediante Wireshark un intercambio de mensajes con el directorio

https://youtu.be/YBV3eNVhxQw

7 Enlace a grabación de pantalla mostrando los programas en funcionamiento

https://youtu.be/V0ZmnHuaMS4

8 Conclusiones

La realización del proyecto nanoFiles ha sido una valiosa experiencia de aprendizaje que nos ha permitido aplicar los conocimientos teóricos adquiridos en la asignatura de redes a un caso práctico de desarrollo. A continuación, se presentan las principales conclusiones extraídas de este trabajo:

8.1 Aprendizaje técnico

- **Diseño de protocolos**

La implementación de un protocolo fiable sobre UDP nos ha permitido comprender en profundidad los mecanismos de control de errores, retransmisiones y timeouts necesarios para garantizar la fiabilidad en un medio no fiable. Por otra parte, el diseño del protocolo P2P sobre TCP ha requerido considerar aspectos como la eficiencia en la transferencia de datos y la gestión de múltiples conexiones simultáneas.

- **Programación multihilo**

El desarrollo del servidor de archivos capaz de atender a múltiples clientes simultáneamente ha supuesto un ejercicio práctico de programación concurrente, permitiéndonos aplicar conceptos como la sincronización y la gestión de recursos compartidos.

- **Arquitecturas híbridas**

La combinación de elementos centralizados (servidor de directorio) y distribuidos (comunicación P2P) nos ha permitido apreciar las ventajas de los sistemas híbridos, que aprovechan lo mejor de ambos paradigmas para ofrecer soluciones más eficientes y escalables.

8.2 Desafíos encontrados

- **Depuración de protocolos**

La identificación y corrección de errores en los protocolos de comunicación ha sido uno de los mayores desafíos. La herramienta Wireshark ha sido fundamental para analizar el intercambio de mensajes y detectar problemas. Así como una saludable cantidad de sentencias try-catch y el uso del debugger de eclipse.

- **Gestión de la concurrencia**

La implementación de un servidor capaz de manejar múltiples conexiones simultáneas y la descarga paralela de fragmentos de archivos desde diferentes servidores han requerido un cuidadoso diseño para evitar condiciones de carrera y garantizar la consistencia de los datos.

- **Integración de componentes**

La coordinación entre los diferentes elementos del sistema (directorio, clientes y servidores) ha exigido un diseño modular y bien estructurado que facilitara la integración y las pruebas del sistema completo.

8.3 Resultados obtenidos

Hemos desarrollado un sistema funcional que cumple con los requisitos básicos especificados e incluye múltiples mejoras, permitiendo la publicación, consulta y descarga de archivos en un entorno distribuido.

La arquitectura diseñada es flexible y modular, permitiendo la extensión del sistema con nuevas funcionalidades sin modificar significativamente los componentes existentes.

8.4 Limitaciones y líneas de trabajo futuro

Tratado con anterioridad.

8.5 Valoración personal

La realización de este proyecto ha sido una experiencia enriquecedora que nos ha permitido consolidar los conocimientos adquiridos en la asignatura y desarrollar habilidades prácticas en el diseño e implementación. Los desafíos encontrados durante el desarrollo han contribuido a mejorar nuestra capacidad de análisis y resolución de problemas, así como nuestras habilidades de trabajo en equipo.

La aproximación práctica a conceptos como el diseño de protocolos, la programación de redes y los sistemas P2P ha complementado de manera significativa los conocimientos teóricos, proporcionándonos una visión más completa y aplicada de la materia.

No obstante, consideramos que el código ya proporcionado podría mejorarse en algunos aspectos: redundancia, nombres variables, uso de estructuras de datos más modernas, clase FileDatabase bastante pobre, clase FileInfo sin métodos de igualdad, verbosidad del código, imposibilidad de mensajes con contenido UTF-8 en la parte UDP del proyecto, entre otros.